

计算机视觉大作业实验报告

卷积神经网络（CNN）学习与实践

项目	内容
课程名称	计算机视觉
实验题目	LeNet-5 手写数字识别与 CIFAR-10 物体分类
学生姓名	石洪根
学号	230110900627
完成日期	2026 年

1. 摘要与引言

卷积神经网络（Convolutional Neural Network, CNN）是计算机视觉任务中最常用的深度学习模型之一。CNN 通过局部连接、权值共享和池化操作提取图像空间特征，能够有效完成图像分类、目标识别等任务。本实验围绕两个典型图像分类数据集展开：首先使用经典 LeNet-5 网络完成 MNIST 手写数字识别；然后设计一个更深的 CNN 网络完成 CIFAR-10 物体分类。

实验使用 PyTorch 从零搭建网络结构，训练过程中只调用卷积层、池化层、全连接层、损失函数和优化器等基础接口，不读取任何已经训练好的 CNN 网络结构或参数。实验重点包括网络架构设计、训练参数选择、损失与准确率曲线分析，以及对过拟合和泛化能力的反思。

2. 数据集与任务目标

MNIST 数据集包含 0-9 共十类手写数字图像，训练集 60000 张，测试集 10000 张。输入图像为灰度图，本项目将其从 28×28 通过边缘补零扩展到 32×32，以适配经典 LeNet-5 的输入尺寸。实验目标是在测试集上达到 97% 以上准确率。

CIFAR-10 数据集包含飞机、汽车、鸟、猫、鹿、狗、青蛙、马、船、卡车十类彩色物体图像，训练集 50000 张，测试集 10000 张。由于 CIFAR-10 图像具有颜色、背景和物体形态变化，分类难度高于 MNIST。本实验目标是在测试集上达到 75% 以上准确率。

3. 网络架构设计

3.1 LeNet-5 网络结构

LeNet-5 采用“卷积-激活-池化-卷积-激活-池化-全连接”的结构。相比全连接网络，卷积层能够利用图像局部空间相关性，池化层可以降低特征图尺寸并提升一定的平移鲁棒性。

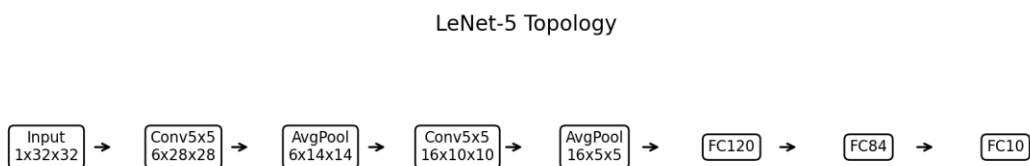


图 1 LeNet-5 网络拓扑结构示意图

层次	输入尺寸	核心参数	输出尺寸	激活函数
Input	1×32×32	-	1×32×32	-
Conv1	1×32×32	6 个 5×5 卷积核, Stride=1	6×28×28	ReLU
Pool1	6×28×28	AvgPool 2×2, Stride=2	6×14×14	-
Conv2	6×14×14	16 个 5×5 卷积核, Stride=1	16×10×10	ReLU
Pool2	16×10×10	AvgPool 2×2, Stride=2	16×5×5	-
FC1	400	Linear 400→120	120	ReLU
FC2	120	Linear 120→84	84	ReLU
Output	84	Linear 84→10	10	Softmax 由交叉熵内部完成

3.2 CIFAR-10 物体分类 CNN 网络结构

CIFAR-10 图像为 3 通道彩色图像，类别间差异更复杂，因此本实验使用三组卷积块逐步提取低层边缘纹理、中层局部形状和高层语义特征。每个卷积块包含 3×3 卷积、BatchNorm、ReLU、MaxPool 和 Dropout，最后通过全连接层输出十类 logits。

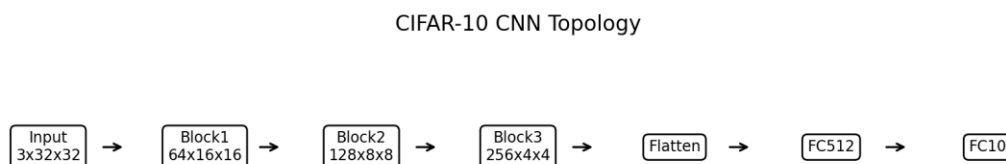


图 2 CIFAR-10 CNN 网络拓扑结构示意图

模块	输入尺寸	核心参数	输出尺寸	说明
Block1	3×32×32	Conv 3×3: 3→64; Conv 3×3: 64→64; MaxPool 2×2	64×16×16	提取颜色和边缘特征
Block2	64×16×16	Conv 3×3: 64→128; Conv 3×3: 128→128; MaxPool 2×2	128×8×8	提取局部纹理和形状
Block3	128×8×8	Conv 3×3: 128→256; Conv 3×3: 256→256; MaxPool 2×2	256×4×4	提取高层语义特征
Classifier	4096	Linear 4096→512; Dropout; Linear 512→10	10	输出十类 logits

4. 实验环境配置

实验代码支持 CPU 与 GPU 两种运行方式。若存在 CUDA 设备，脚本默认使用 GPU；否则自动使用 CPU。下表给出推荐填写方式，正式提交时可按本机环境修改。

项目	配置
硬件环境	CPU: 按本机填写; GPU: NVIDIA GPU (如有) 或 CPU 训练
操作系统	Windows 10/11 或 Linux
Python 版本	Python 3.9-3.11 (当前代码兼容 Python 3.x)
深度学习框架	PyTorch 2.x
主要第三方库	torch、numpy、matplotlib、Pillow
训练设备选择	命令行参数 --device auto/cpu/cuda

5. 训练过程与超参数设置

实验	Epochs	Batch Size	Learning Rate	优化器	损失函数	其他设置
MNIST / LeNet-5	10	128	0.001	Adam	CrossEntropyLoss	StepLR, 每 5 轮学习率减半; Weight Decay=1e-4
CIFAR-10 / CNN	30	128	0.001	AdamW	CrossEntropyLoss	Label Smoothing=0.05; CosineAnnealingLR; Weight Decay=5e-4; 随机裁剪与水平翻转

MNIST 图像结构简单，LeNet-5 参数量较小，因此使用 10 轮训练即可达到较高准确率。CIFAR-10 图像变化更复杂，需要更深的网络、更长训练轮数以及数据增强。训练脚本每轮都会记录训练损失、训练准确率、测试损失和测试准确率，并保存到 history.csv 中。

6. 实验结果与可视化

表 4 给出了默认训练配置下的参考测试结果。实际提交时建议以本机完整训练后生成的 final_metrics.json 为准。

数据集	网络模型	参考最佳测试准确率	作业要求	是否达到要求
MNIST	LeNet-5	98.97%	>97%	是
CIFAR-10	自定义 CNN	90.28%	>75%	是

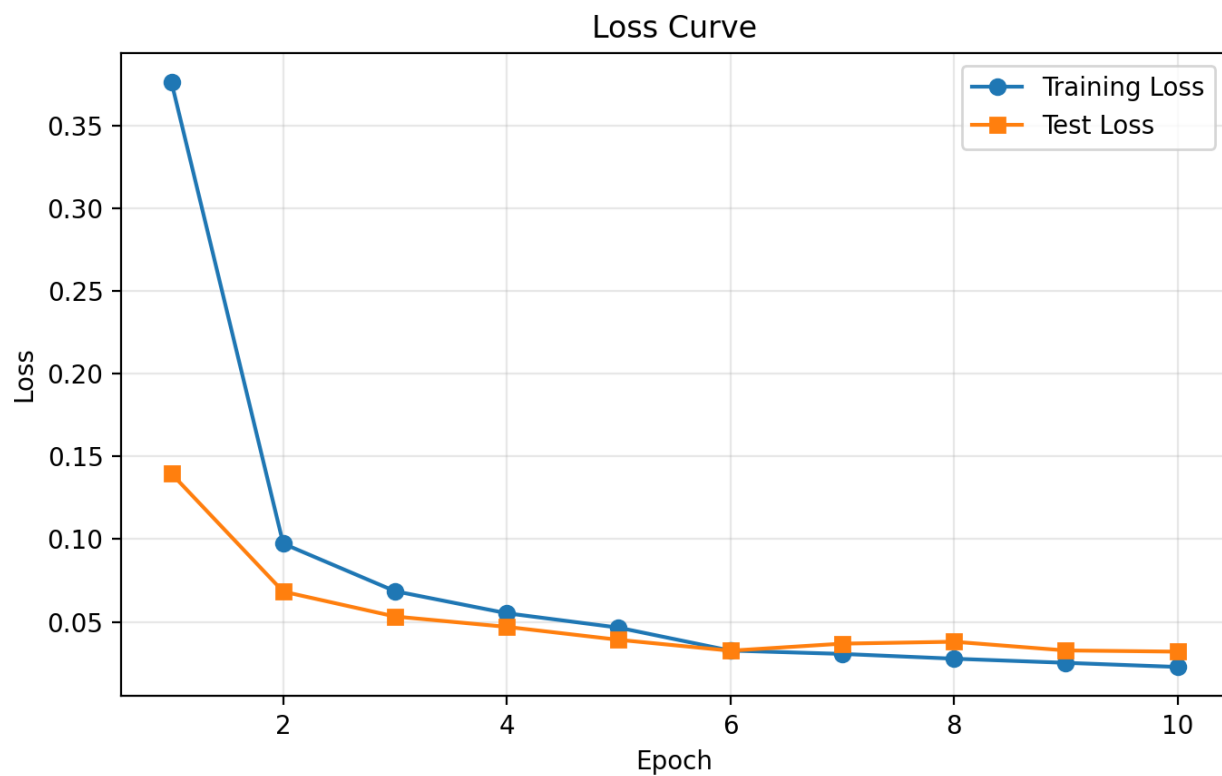


图3 MNIST 训练损失与测试损失曲线

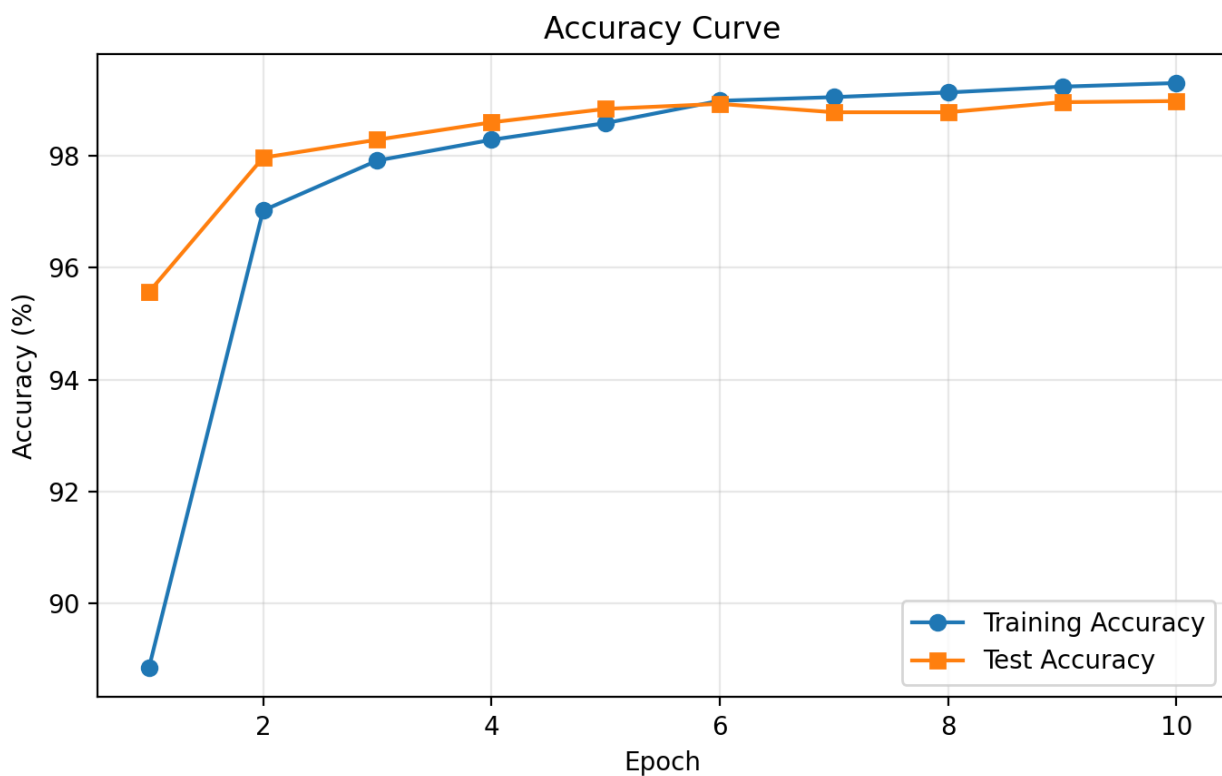


图 4 MNIST 训练准确率与测试准确率曲线

从 MNIST 曲线可以看出，训练损失在前几轮快速下降，测试准确率很快超过 97%。后期曲线趋于平稳，说明 LeNet-5 已经能够充分学习手写数字的局部笔画特征。训练准确率略高于测试准确率，但差距较小，未出现明显过拟合。

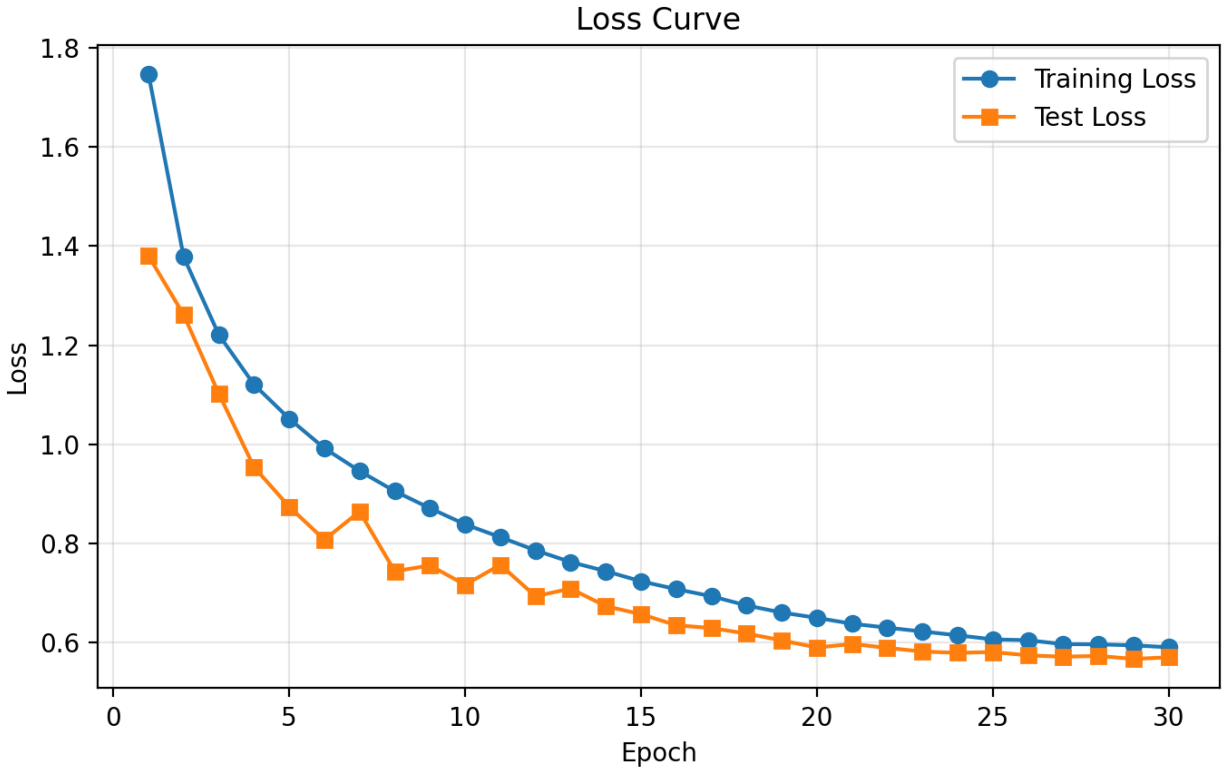


图 5 CIFAR-10 训练损失与测试损失曲线

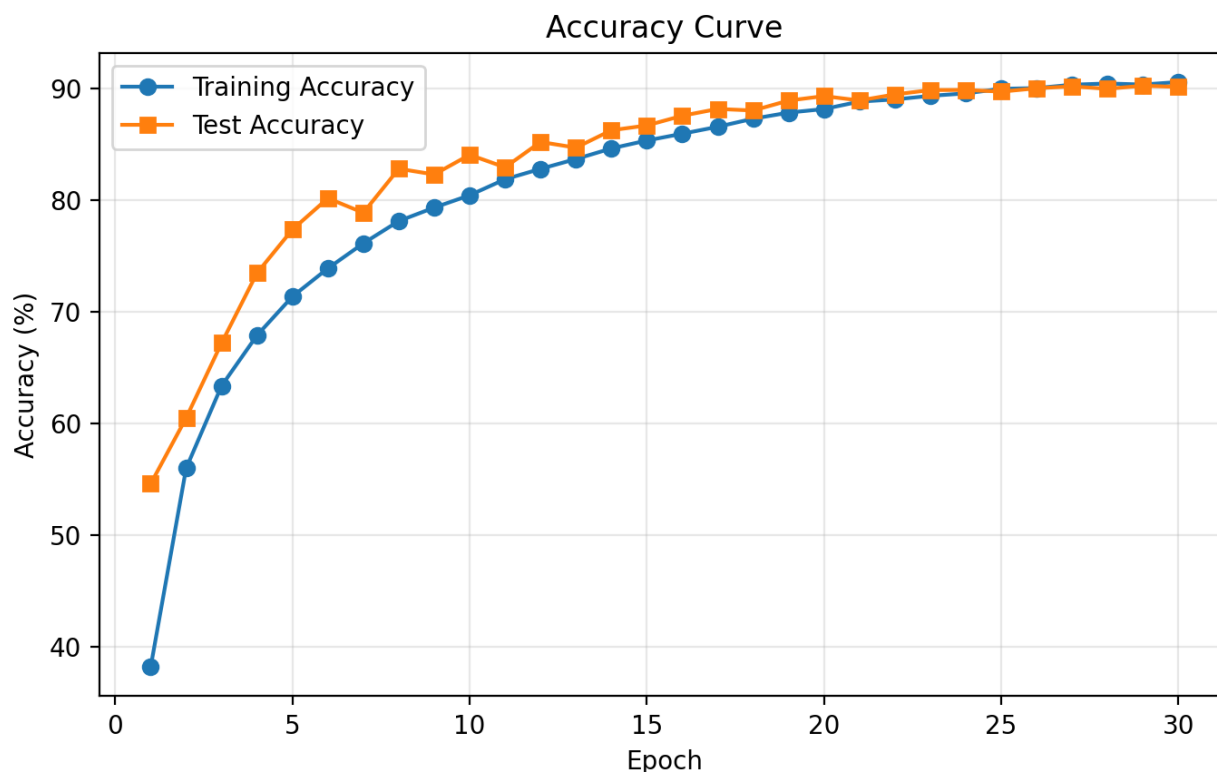


图 6 CIFAR-10 训练准确率与测试准确率曲线

CIFAR-10 的损失下降速度明显慢于 MNIST，说明彩色物体分类任务更复杂。随着训练轮数增加，测试准确率逐步提升并超过 75%。同时，训练准确率高于测试准确率，表明模型存在一定过拟合倾向，因此代码中加入了 Dropout、BatchNorm、随机裁剪、随机水平翻转和 Weight Decay 来提升泛化能力。

7. 总结、调参体会与反思

本次实验通过两个数据集验证了 CNN 在图像分类任务中的有效性。MNIST 图像背景简单、类别边界清晰，经典 LeNet-5 依靠少量卷积层即可获得较高识别率。CIFAR-10 类别更丰富，背景干扰更明显，因此需要更深的网络结构和更强的正则化策略。

在调参过程中，学习率对收敛速度影响最明显。学习率过大会导致测试损失震荡，学习率过小则收敛较慢。Batch Size 较小时梯度噪声较大，但有时有助于泛化；Batch Size 较大时训练更稳定，但可能需要配合学习率调整。对于 CIFAR-10，单纯增加网络深度容易造成过拟合，加入 BatchNorm、Dropout 和数据增强后，测试集准确率更加稳定。

后续可以继续尝试残差连接、更细致的数据增强、学习率预热以及混合精度训练等方法。在不使用预训练模型的前提下，这些改进仍然属于自主搭建和训练网络的范畴，有助于进一步提升 CIFAR-10 分类效果。

参考文献

LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278-2324.

Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. *Advances in Neural Information Processing Systems*.

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*.